Project 11: Log File Analyser for Intrusion Detection

Name: Mohammed Fadil PK

Domain: Cybersecurity / Security Operations (Blue Team)

Date: March 4, 2026

1. Introduction

In cybersecurity, system logs are the primary source of truth for detecting malicious activity. However, manually reviewing thousands of log lines is inefficient. This project automates threat hunting by parsing Linux SSH authorisation logs to detect patterns indicative of brute-force attacks.

2. Abstract

The "Log File Analyser" is a Blue Team Python utility that serves as an Intrusion Detection System (IDS). It reads standard Linux auth logs, uses Regular Expressions (Regex) to extract attacker IP addresses, and flags IPs that repeatedly fail to authenticate. It then generates an automated visual dashboard using Matplotlib to help analysts quickly identify high-risk threats.

3. Tools & Technologies Used

- **Operating System:** Kali Linux.
- **Language:** Python 3.
- **Libraries:** re (Regex for text parsing), collections.Counter (for data aggregation), matplotlib.pyplot (for data visualisation).

4. Steps Involved in Building the Project

Step 1: Environment Setup & Log Simulation

- Configured the Python environment with required data science libraries (matplotlib, pandas).
- Created a simulated /var/log/auth.log file containing both legitimate logins and a simulated brute-force attack to safely test the tool.

Step 2: Developing the Parsing Logic

- Implemented the analyze_ssh_logs() function.
- Utilised the Regex pattern r"Failed password for .* from ([0-9]+\.[0-9]+\.[0-9]+\.[0-9]+)" to isolate the specific IP addresses responsible for failed login attempts while ignoring successful authentications.

Step 3: Threshold Alerting

- Implemented a threshold mechanism: IP addresses with 3 or more failed attempts trigger a critical **Brute-Force Alert**, while fewer attempts trigger a standard warning.

Step 4: Data Visualisation

- Integrated matplotlib to dynamically generate a bar chart.
- Programmed conditional colouring (Red for critical threats, Orange for warnings) to create an intuitive dashboard for security analysts.

5. Results (Evidence)

Screenshot 1: Terminal Output & Threat Detection

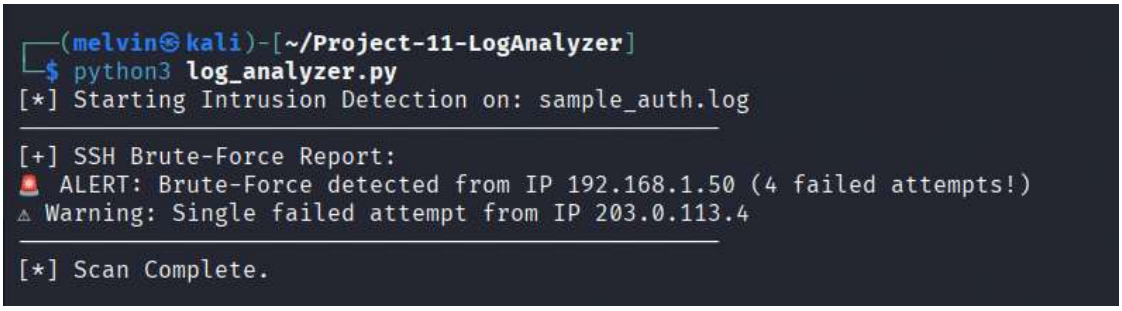

Figure 1: *Terminal execution showing the script successfully ignoring valid logins, issuing a warning for a single failure, and flagging 192.168.1.50 with a Brute-Force Alert.*

Screenshot 2: Visual Threat Dashboard

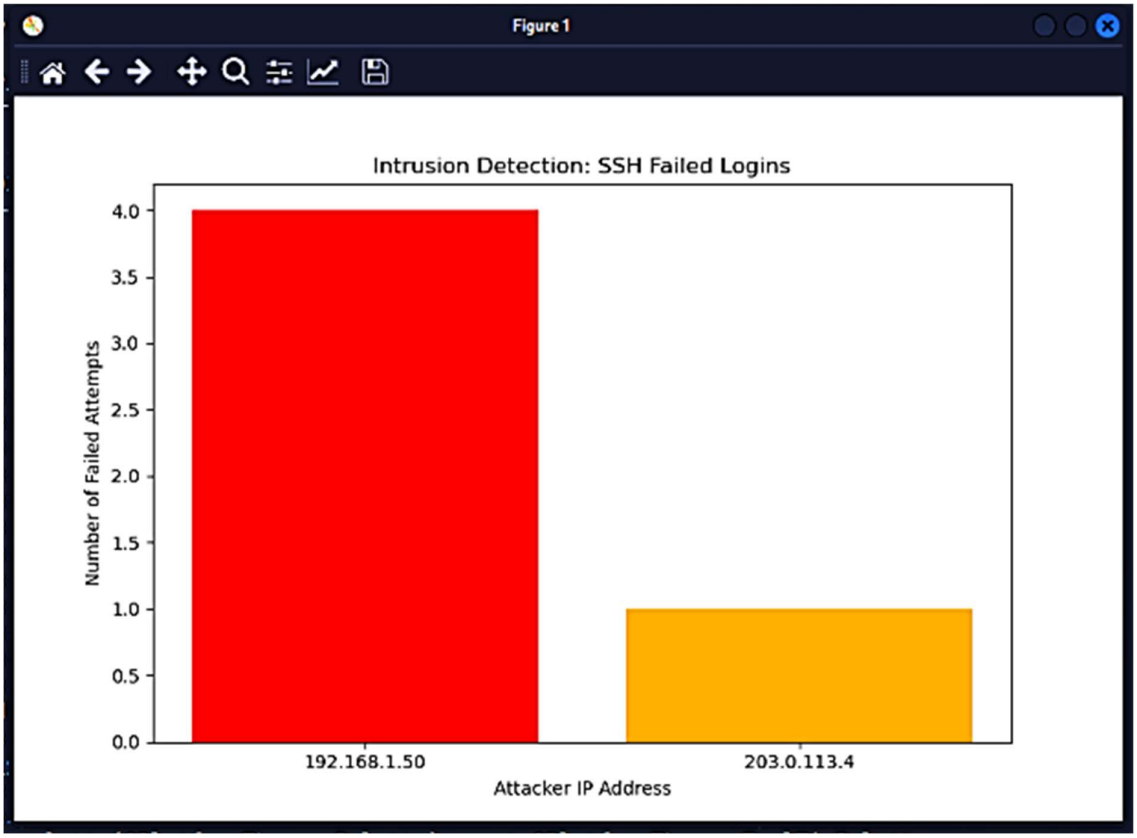

Figure 2: *Automated generation of the threat intelligence bar chart, visually representing the volume of attacks per IP address.*

6. Conclusion

This project successfully demonstrates how Python can be utilised to automate Blue Team defensive operations. By transforming raw, unstructured log data into clear, actionable visual intelligence, security analysts can drastically reduce incident response times. Future iterations of this tool could include automatic IP blacklisting using iptables or cross-referencing attacker IPs with public threat databases.